CS F320 Foundations of Data Science

Assignment 1: Polynomial, Ridge, Lasso and Bayesian Regression

Chaitanya Keyal 2023B4A70727H
Abhishek Verma 2023B4A71256H
Palash Jain 2023A2B51271H

October 2026

Implementation

Everything is written from scratch in Python 3.12 with NumPy for linear algebra and Matplotlib for plots; no fitting library is used. The code, the data and every output are at

`https://github.com/Chaitanya-Keyal/polynomial-regression`

`src/p1.py` to `src/p4.py` hold one problem each, `src/common/` the shared bases and solvers, and `python -m src.run` regenerates every figure and every number in this report from the six provided CSV files, which are loaded as given. $\sin(2\pi x)$ appears only in plots. The random seed for the per-epoch shuffling in P1 is 42 (a fresh `np.random.default_rng(42)` per run), so a rerun reproduces every number exactly. The few values that rounding alone determines (the converged E near 10^{-30} and the gaps between GD and the closed form in P1 and P4) are printed to one significant figure, since another linear-algebra library can change their leading digit. No explicit matrix inverse is formed: least squares uses the SVD-based `np.linalg.lstsq`, and ridge and the Bayesian posterior use `np.linalg.solve`. RMSE is $\sqrt{\text{mean squared error}}$ throughout.

1 P1: Closed form vs. the gradient descent family

Method. The degree-d model $y(x, \mathbf{w}) = \sum_{j=0}^{d} w_j x^j$ has design matrix $\mathbf{\Phi} \in \mathbb{R}^{N\times(d+1)}$ with rows $[1, x_n, \ldots, x_n^d]$ and objective $E(\mathbf{w}) = \frac{1}{2}\|\mathbf{\Phi}\mathbf{w} - \mathbf{t}\|^2$. The closed-form minimiser solves $\mathbf{\Phi}^\top\mathbf{\Phi}\mathbf{w} = \mathbf{\Phi}^\top\mathbf{t}$, but for $d = 9$ we have $\text{cond}(\mathbf{\Phi}) = 1.52 \times 10^7$, and forming $\mathbf{\Phi}^\top\mathbf{\Phi}$ would square that to about 10^{14}. We therefore solve the least squares problem directly with `np.linalg.lstsq`, which works through the SVD of $\mathbf{\Phi}$.

For gradient descent we take the thin QR decomposition $\mathbf{\Phi} = QR$, set $\boldsymbol{\alpha} = R\mathbf{w}$ and minimise $E(\boldsymbol{\alpha}) = \frac{1}{2}\|Q\boldsymbol{\alpha} - \mathbf{t}\|^2$ from $\boldsymbol{\alpha} = \mathbf{0}$, then recover $\mathbf{w} = R^{-1}\boldsymbol{\alpha}$ by a solve with R ($\text{cond}(Q) = 1.000$). With $\mathbf{q}_n^\top$ the n-th row of Q and $r_n = \mathbf{q}_n^\top\boldsymbol{\alpha} - t_n$ its residual, one epoch is

- **Batch GD**: one step $\boldsymbol{\alpha} \leftarrow \boldsymbol{\alpha} - \eta\, Q^\top(Q\boldsymbol{\alpha} - \mathbf{t})$, the full sum, not divided by N;
- **Stochastic GD**: shuffle, then for each n in turn $\boldsymbol{\alpha} \leftarrow \boldsymbol{\alpha} - \eta\, r_n\mathbf{q}_n$ ($N = 10$ updates);
- **Mini-batch GD**: shuffle, then for each batch B of the shuffled order (sizes 3, 3, 3, 1) $\boldsymbol{\alpha} \leftarrow \boldsymbol{\alpha} - \frac{\eta}{|B|}\sum_{n\in B} r_n\mathbf{q}_n$ (4 updates).

Closed-form fits. Figure 1 shows the least-squares fits for $d = 1, \ldots, 9$ and Table 1 their training error. The models are nested, so the training error can only fall as d grows; at $d = 9$ there are as many weights as points and the polynomial interpolates the data ($E \approx 0$). Degrees 1 and 2 underfit, degrees 3 to 6 follow $\sin(2\pi x)$ closely, and from $d = 7$ on the curve bends between the points to chase the noise, most visibly near the ends of $[0, 1]$.

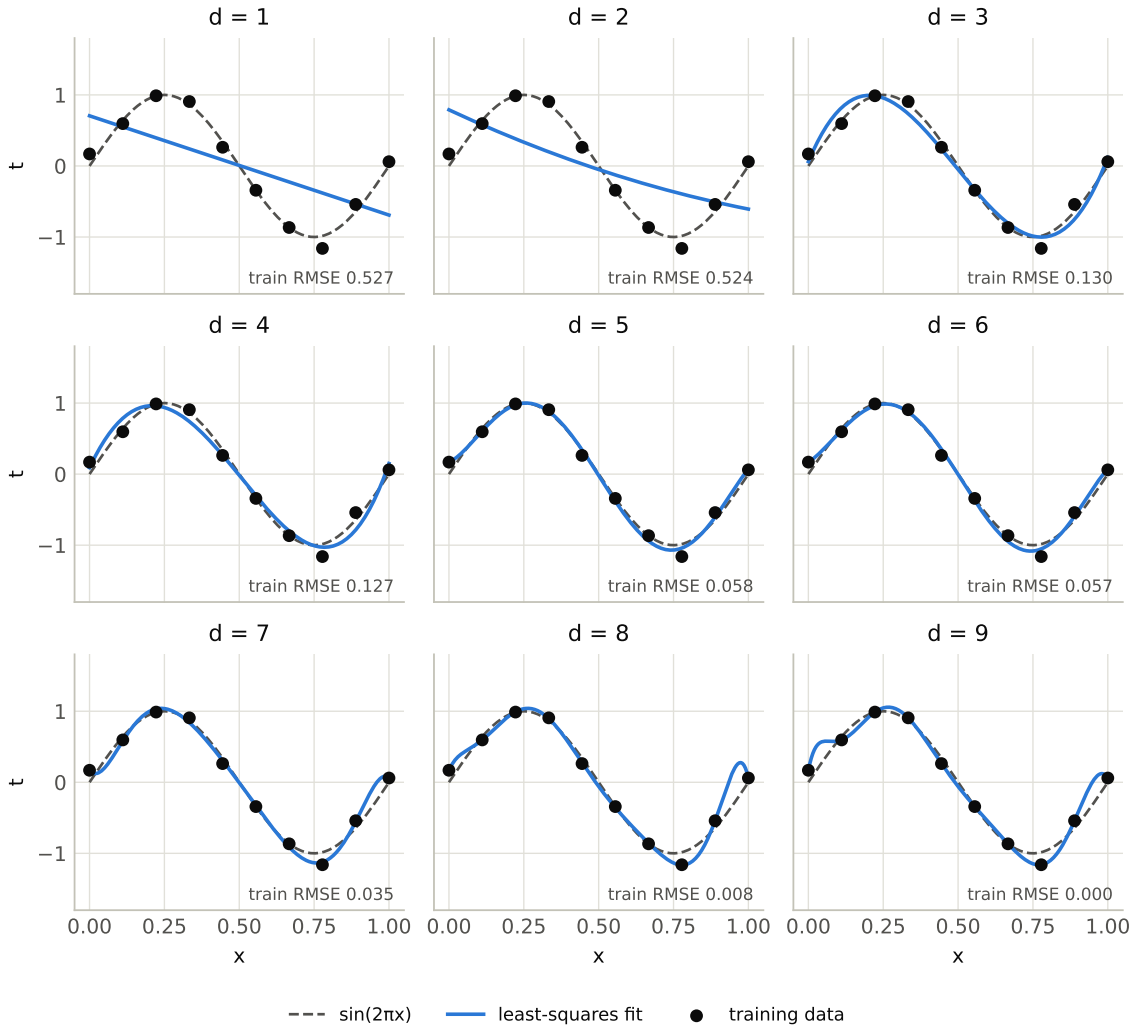


Figure 1: Least-squares polynomial fits of degree 1 to 9 to the $N = 10$ training points, with the true function $\sin(2\pi x)$.

d	training $E(\mathbf{w})$	training RMSE
1	1.39	0.527
2	1.37	0.524
3	0.0844	0.130
4	0.0808	0.127
5	0.0166	0.0575
6	0.0161	0.0568
7	0.00627	0.0354
8	3.09×10^{-4}	0.00787
9	7×10^{-23}	4×10^{-12}

Table 1: Training error of the closed-form fits.

Loss curves. Figure 2 shows E per epoch for the three variants at $d = 9$ and $\eta = 0.1$ over all 2000 epochs of the verification run: every curve has flattened at the floating-point floor (Batch near 10^{-30}, SGD and Mini-batch near 10^{-29}, since their 10 and 4 rounded updates per epoch leave more rounding). $E < 10^{-12}$ is reached after 136 epochs by Batch GD, 136 by Stochastic GD and 353 by Mini-batch GD; over the first 100 epochs E shrinks per epoch by an average factor of 0.8100, 0.8100 and 0.9242 respectively.

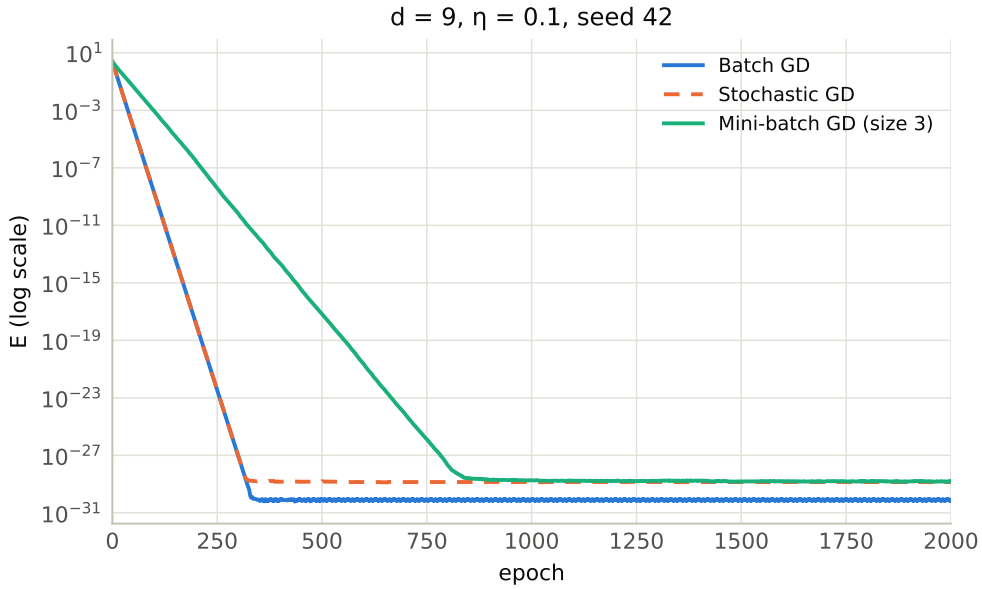


Figure 2: E vs. epoch for Batch, Stochastic and Mini-batch GD ($d = 9$, $\eta = 0.1$, seed 42). The SGD curve (dashed) lies on the Batch curve until both reach the floating-point floor; below it only rounding separates them.

Learning-rate study. In the QR coordinates the Hessian of E is $Q^\top Q = I$, so every eigenvalue is 1 and Batch GD gives $\boldsymbol{\alpha}_k - \boldsymbol{\alpha}^\star = (1-\eta)^k(\boldsymbol{\alpha}_0 - \boldsymbol{\alpha}^\star)$ and $E_k = (1-\eta)^{2k}E_0$ with $E_0 = \frac{1}{2}\|\mathbf{t}\|^2 = 2.381$. GD converges exactly when $|1-\eta| < 1$, i.e. $0 < \eta < 2$, fastest at $\eta = 1$ (one epoch). Figure 3 shows the three regimes over 100 epochs:

- $\eta = 0.01$ (too small): E is multiplied by only $0.99^2 \approx 0.98$ per epoch and is still 0.3190 after 100 epochs;
- $\eta = 0.5$ (well chosen): E falls by a factor of 4 per epoch and reaches the floating-point floor within about 50 epochs ($E_{100} = 5 \times 10^{-32}$);
- $\eta = 2.1$ (too large): $|1-\eta| = 1.1$, so the error flips sign and grows by a factor of 1.1 every epoch. After 100 epochs $E = 4.521 \times 10^8 = 1.1^{200}E_0$, and $\boldsymbol{\alpha}_{100} = (1 - 1.1^{100})\boldsymbol{\alpha}^\star$, so the weight vector is -13780 times the closed-form solution (Table 2).

At exactly $\eta = 2$ the factor is -1: $\boldsymbol{\alpha}$ alternates between $\mathbf{0}$ and $2\boldsymbol{\alpha}^\star$, each step overshooting the minimum by exactly as much as it started from it, and E stays at E_0 forever. It neither converges nor diverges.

j	$w_{\text{closed},j}$	$w_{100,j}$ at $\eta = 2.1$
0	0.169	-2329
1	21.3	-2.93×10^5
2	-414	5.70×10^6
3	3788	-5.22×10^7
4	-17589	2.42×10^8
5	45606	-6.28×10^8
6	-69409	9.56×10^8
7	61610	-8.49×10^8
8	-29469	4.06×10^8
9	5856	-8.07×10^7

Table 2: The closed-form weights for $d = 9$, and the weights Batch GD reaches after 100 epochs with $\eta = 2.1$: the same vector scaled by $1 - 1.1^{100}$.

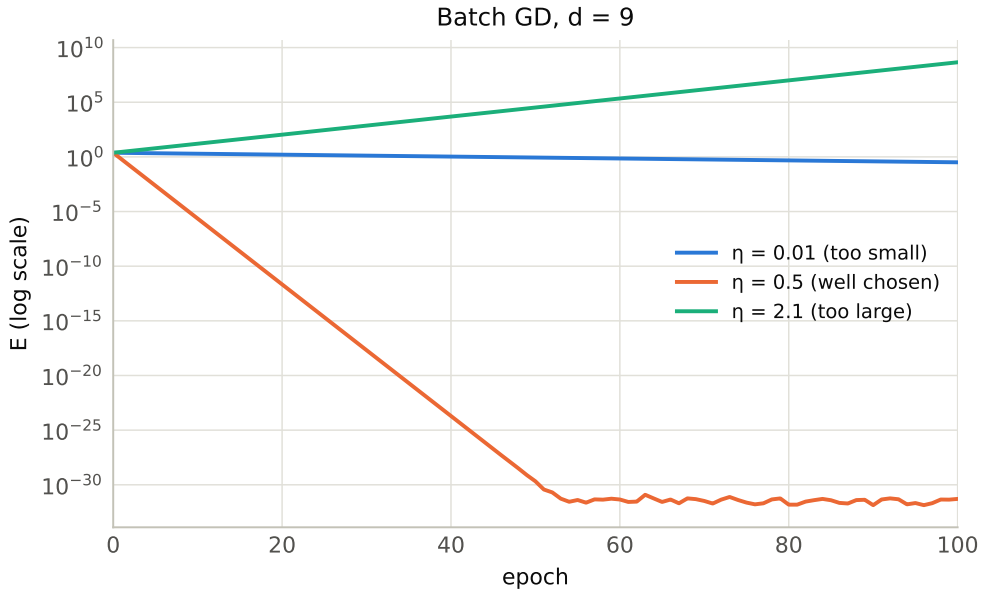


Figure 3: Batch GD ($d = 9$, QR-reparametrised) for a learning rate that is too small, well chosen and too large.

Verification. After 2000 epochs at $\eta = 0.1$:

variant	$\|\mathbf{w}_{\mathrm{GD}} - \mathbf{w}_{\mathrm{closed}}\|_2$	relative error	final E
Batch GD	4×10^{-6}	4×10^{-11}	7×10^{-31}
Stochastic GD	4×10^{-6}	4×10^{-11}	1×10^{-29}
Mini-batch GD	4×10^{-6}	4×10^{-11}	2×10^{-29}

Table 3: GD against the closed form, $d = 9$; $\|\mathbf{w}_{\mathrm{closed}}\|_2 = 1.09 \times 10^5$.

The weights are large (Table 2), so an absolute error of this size is a relative error of only 4×10^{-11}: all three variants agree with the closed form to the precision the problem allows. The error is not optimisation error, since GD converges to the QR solution $R^{-1}Q^{\top}\mathbf{t}$ to within rounding; what remains is the gap between two direct solutions of the same system (QR, and the SVD inside `lstsq`), well inside the relative error of about $\mathrm{cond}(\mathbf{\Phi})\,\epsilon_{\mathrm{mach}} \approx 3 \times 10^{-9}$ that rounding allows.

Discussion. In Figure 2 the SGD curve lies on the Batch curve epoch for epoch (relative gap below 7×10^{-11} while $E > 10^{-10}$) until both reach the rounding floor, while Mini-batch GD is clearly slower. (i) With $N = d + 1 = 10$ the thin Q is square and orthogonal, so its *rows* $\mathbf{q}_n$ are orthonormal: an SGD step on point n changes residual r_m by $-\eta r_n \mathbf{q}_m^{\top}\mathbf{q}_n$, which multiplies r_n by $(1 - \eta)$ and leaves every other residual untouched, so one pass in any order multiplies every residual by $(1 - \eta)$ once, exactly like one Batch step $\mathbf{r} \leftarrow (I - \eta QQ^{\top})\mathbf{r} = (1 - \eta)\mathbf{r}$. (ii) Mini-batch GD *averages* over its batch, so the nine points in batches of three shrink only by $(1 - \eta/3)$ per epoch (the singleton by $1 - \eta$): E shrinks by a factor of about 0.9242 per epoch instead of 0.81, so it needs about 2.6 times as many epochs (353 vs. 136).

2 P2: Ridge regression as a MAP estimate

MAP $\Rightarrow$ ridge. By Bayes' rule $p(\mathbf{w} \mid \mathbf{t}) \propto p(\mathbf{t} \mid \mathbf{w})\, p(\mathbf{w})$ with $p(\mathbf{t} \mid \mathbf{w}) \propto \exp(-\frac{\beta}{2}\|\mathbf{\Phi}\mathbf{w} - \mathbf{t}\|^2)$ and $p(\mathbf{w}) \propto \exp(-\frac{\alpha}{2}\|\mathbf{w}\|^2)$. Taking $-\log$ and dividing by β, maximising the posterior is minimising

$$\tilde{E}(\mathbf{w}) = \tfrac{1}{2}\|\mathbf{\Phi}\mathbf{w} - \mathbf{t}\|^2 + \tfrac{\lambda}{2}\|\mathbf{w}\|^2, \qquad \lambda = \alpha/\beta.$$

Setting $\nabla\tilde{E} = \mathbf{\Phi}^\top(\mathbf{\Phi}\mathbf{w} - \mathbf{t}) + \lambda\mathbf{w} = \mathbf{0}$ gives $(\mathbf{\Phi}^\top\mathbf{\Phi} + \lambda I)\mathbf{w} = \mathbf{\Phi}^\top\mathbf{t}$. For $\lambda > 0$ the matrix is symmetric positive definite, so the solution is unique; we solve the system directly for every λ in the grid (49 values from 10^{-10} to 10^{2}), in the $d = 9$ monomial basis of P1.

GD at $\lambda = 10^{-2}$. Plain batch GD in the monomial basis, $\mathbf{w} \leftarrow \mathbf{w} - \eta\,(\mathbf{\Phi}^\top(\mathbf{\Phi}\mathbf{w} - \mathbf{t}) + \lambda\mathbf{w})$ from $\mathbf{w} = \mathbf{0}$ with $\eta = 0.04$, reached $\|\mathbf{w}_{\mathrm{GD}} - \mathbf{w}_{\mathrm{closed}}\|/\|\mathbf{w}_{\mathrm{closed}}\| < 10^{-6}$ after 28,227 iterations (Figure 4); its final objective 0.4840278 matches the closed form's 0.4840278. Here $\mathrm{cond}(\mathbf{\Phi}^\top\mathbf{\Phi}) = 2.31 \times 10^{14}$ and $\mathrm{cond}(\mathbf{\Phi}^\top\mathbf{\Phi} + \lambda I) = 2038$: the penalty adds λ to every eigenvalue, lifting the smallest from almost zero to about λ and cutting the condition number by eleven orders of magnitude, which is what makes plain GD in this basis practical.

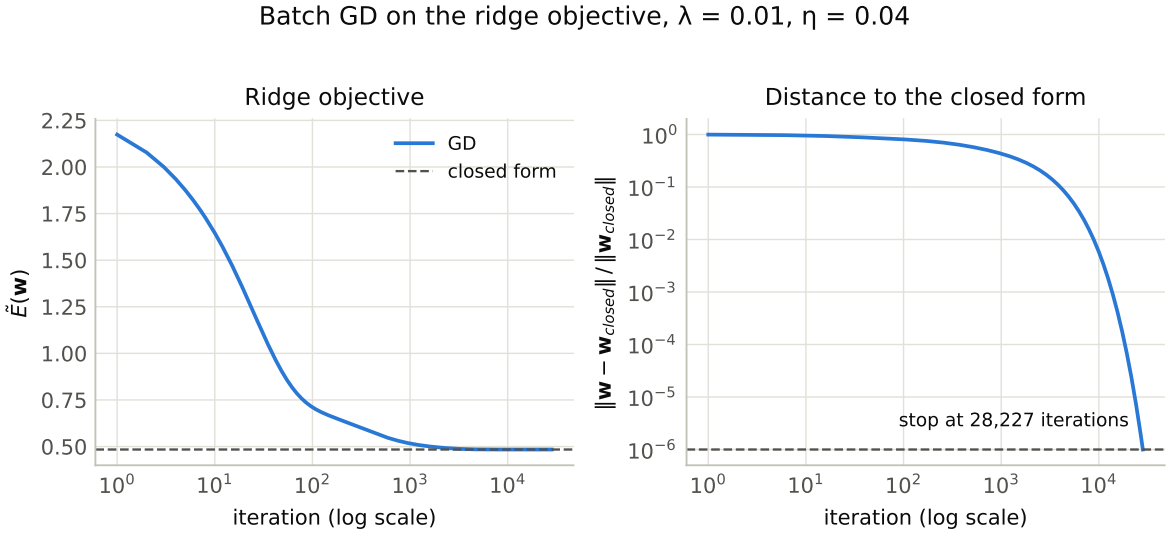


Figure 4: Batch GD on the ridge objective at $\lambda = 10^{-2}$: $\tilde{E}(\mathbf{w})$ per iteration (left) and the relative distance to the closed-form solution (right).

RMSE sweep and $\lambda^\star$. Figure 5 shows train and test RMSE over the grid. The test RMSE is smallest at $\lambda^\star = 1.78 \times 10^{-7}$, where it is 0.1146 (train 0.05410). It is within 1.05× its minimum on the 8 contiguous grid points $\lambda \in [3.16 \times 10^{-8}, 1.78 \times 10^{-6}]$ around $\lambda^\star$, which is the interval we report; after peaking at 1.066× the minimum at $\lambda = 10^{-5}$ it dips back under 1.05× at the isolated grid point $\lambda = 10^{-4}$, so the qualifying set is not a single interval. The minimum is flat: the test RMSE stays within 5% of its minimum over almost two decades of λ around $\lambda^\star$ and within 7% all the way from 3.16×10^{-8} to 10^{-4}, so the exact choice of λ in this range hardly matters. Below the interval the penalty is too weak and the degree-9 polynomial chases the noise (at $\lambda = 10^{-10}$ the train RMSE falls to 0.01649 while the test RMSE climbs to 0.1605), and above it the test RMSE first plateaus near 0.12 until $\lambda \approx 10^{-4}$, after which the over-shrunk weights underfit and train and test RMSE rise together (to 0.2660 at $\lambda = 10^{-2}$).

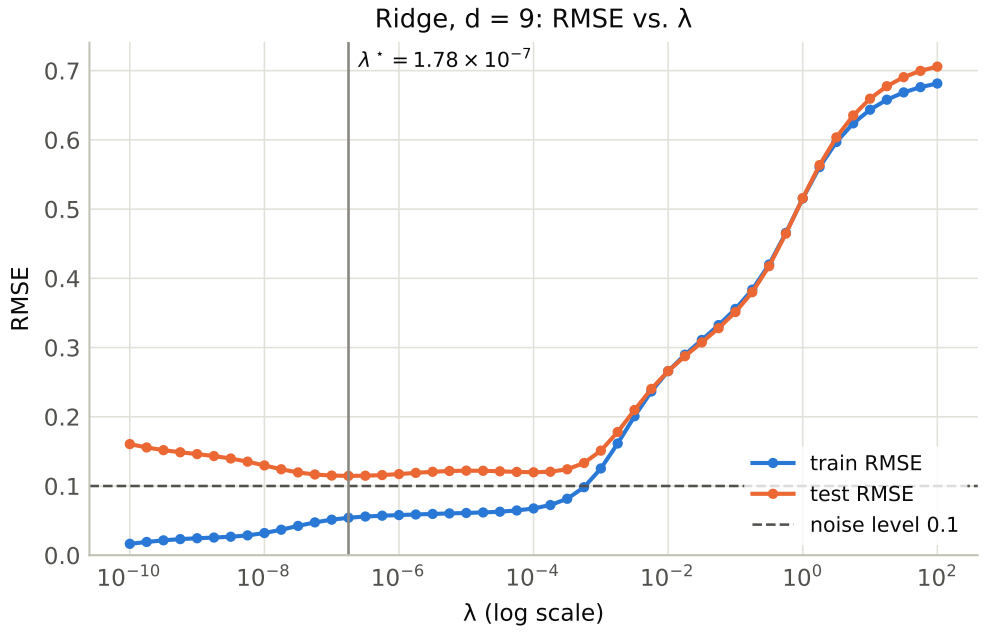


Figure 5: Train and test RMSE of ridge regression ($d = 9$) vs. λ, with the noise level $\sqrt{0.01} = 0.1$.

Fitted curves. Figure 6 shows the fits at the three values of λ, and Table 4 their weight norms. The norm falls by almost three orders of magnitude from the overfit to the underfit model: the penalty buys smoothness by forbidding the large, cancelling coefficients an interpolating polynomial needs.

	λ	$\|\mathbf{w}\|_2$	train RMSE	test RMSE
overfit	10^{-10}	3979	0.01649	0.1605
$\lambda^\star$	1.78×10^{-7}	124	0.05410	0.1146
underfit	10^{-2}	5.11	0.2659	0.2660

Table 4: The three ridge fits of Figure 6.

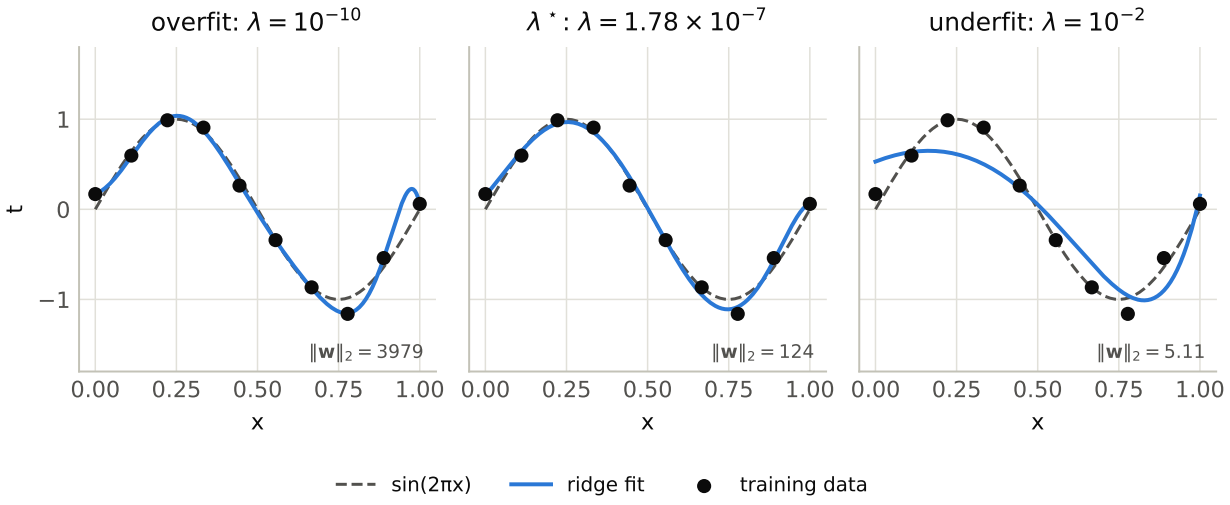


Figure 6: Ridge fits ($d = 9$) for $\lambda = 10^{-10}$, for $\lambda^\star$ and for $\lambda = 10^{-2}$, with the training data and $\sin(2\pi x)$.

3 P3: L1-regularised regression via subgradient descent

Method. With $u = 2x - 1$, the design matrix is $\Phi_{nj} = P_j(u_n)$, $j = 0, \ldots, 9$ (NumPy's `legvander` applied to $2x - 1$). It spans the same degree-9 polynomials as the monomial basis, but $\mathrm{cond}(\boldsymbol{\Phi}^\top \boldsymbol{\Phi}) = 611$ instead of 2.31×10^{14} for the monomials (Section 2). Subgradient descent on $E(\mathbf{w}) = \frac{1}{2}\|\boldsymbol{\Phi}\mathbf{w} - \mathbf{t}\|^2 + \lambda\|\mathbf{w}\|_1$ runs $\mathbf{w} \leftarrow \mathbf{w} - \eta\,(\boldsymbol{\Phi}^\top(\boldsymbol{\Phi}\mathbf{w} - \mathbf{t}) + \lambda\,\mathrm{sign}(\mathbf{w}))$ with $\mathrm{sign}(0) = 0$ and $\eta = 0.05$ from $\mathbf{w} = \mathbf{0}$ for 20,000 iterations at each of the 21 values of λ from 10^{-4} to 1. Every reported quantity comes from the final iterate, and a coefficient counts as ≈ 0 when $|w_j| < 2\eta\lambda$, since with a constant step a zero coefficient chatters around 0 with amplitude about $\eta\lambda$.

Convergence and verification at $\lambda = 0.1$. Figure 7 shows E and the best E so far. Subgradient descent is not a descent method: once near the optimum, the sign term flips for the coefficients at zero and E rises and falls by about 10^{-3} (right panel), while the best-so-far E steps down to about 10^{-4} above $E^\star$ and then levels off: with a constant step it cannot close that gap. The final iterate has $E = 0.2636$, against the reference $E^\star = 0.2625$, and

$$\mathbf{w} = (-0.006,\ -0.930,\ 0.043,\ 1.115,\ 0.030,\ -0.203,\ -0.002,\ -0.025,\ 0.002,\ 0.001),$$
$$\mathbf{w}^\star = (0.000,\ -0.928,\ 0.045,\ 1.117,\ 0.032,\ -0.201,\ 0.000,\ -0.026,\ 0.000,\ 0.000),$$

so $\|\mathbf{w} - \mathbf{w}^\star\|_2 = 0.00764$, the size of the $\eta\lambda$ chatter. The optimality conditions hold to within that chatter: with $\mathbf{r} = \mathbf{t} - \boldsymbol{\Phi}\mathbf{w}$, over the 6 coefficients that are not ≈ 0, $\max_j |\boldsymbol{\Phi}_j^\top \mathbf{r} - \lambda\,\mathrm{sign}(w_j)| = 0.0144$, and over the 4 that are, $\max_j |\boldsymbol{\Phi}_j^\top \mathbf{r}|/\lambda = 0.683 \le 1$.

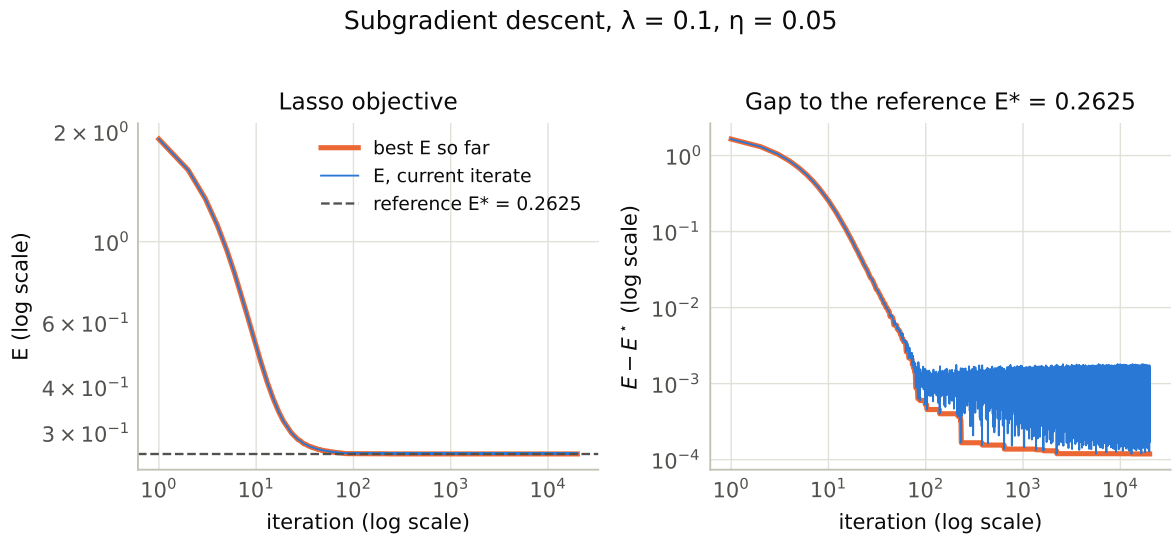


Figure 7: Subgradient descent at $\lambda = 0.1$: E and the best E so far (left), and both minus the reference $E^\star$ (right). The iteration axis is logarithmic so that the fast first 100 iterations are visible.

RMSE sweep and $\lambda^\star$. Figure 8 shows train and test RMSE. The test RMSE is smallest at $\lambda^\star = 10^{-1}$, where it is 0.09854; there 4 coefficients are ≈ 0, with indices $j \in \{0, 6, 8, 9\}$, exactly the zeros of $\mathbf{w}^\star$.

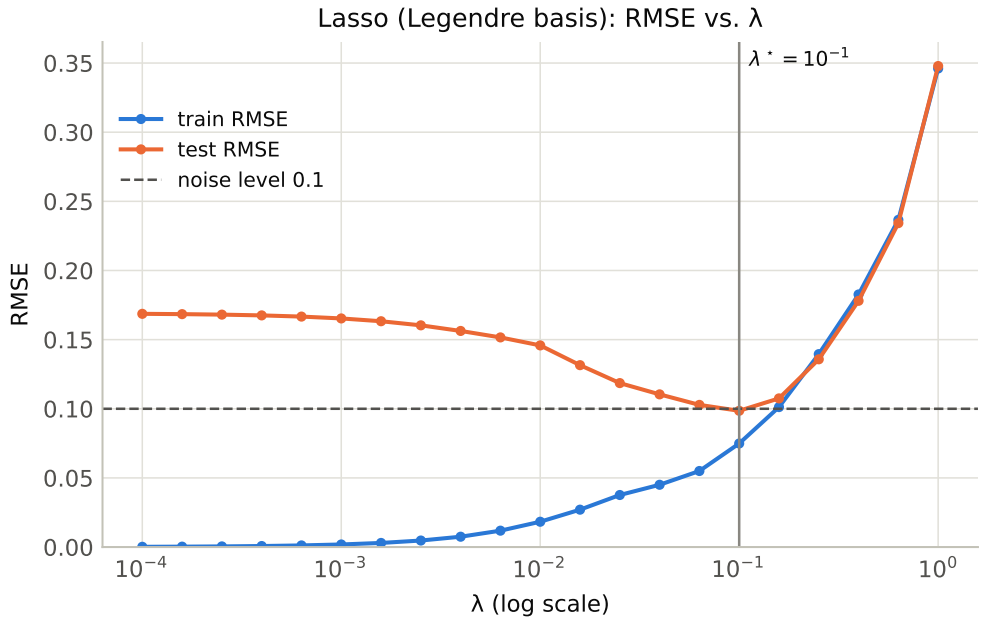


Figure 8: Train and test RMSE of the Lasso (Legendre basis) vs. λ, with the noise level 0.1.

Coefficient path. Figure 9 shows every w_j against λ, and Table 5 how many are ≈ 0 at each λ.

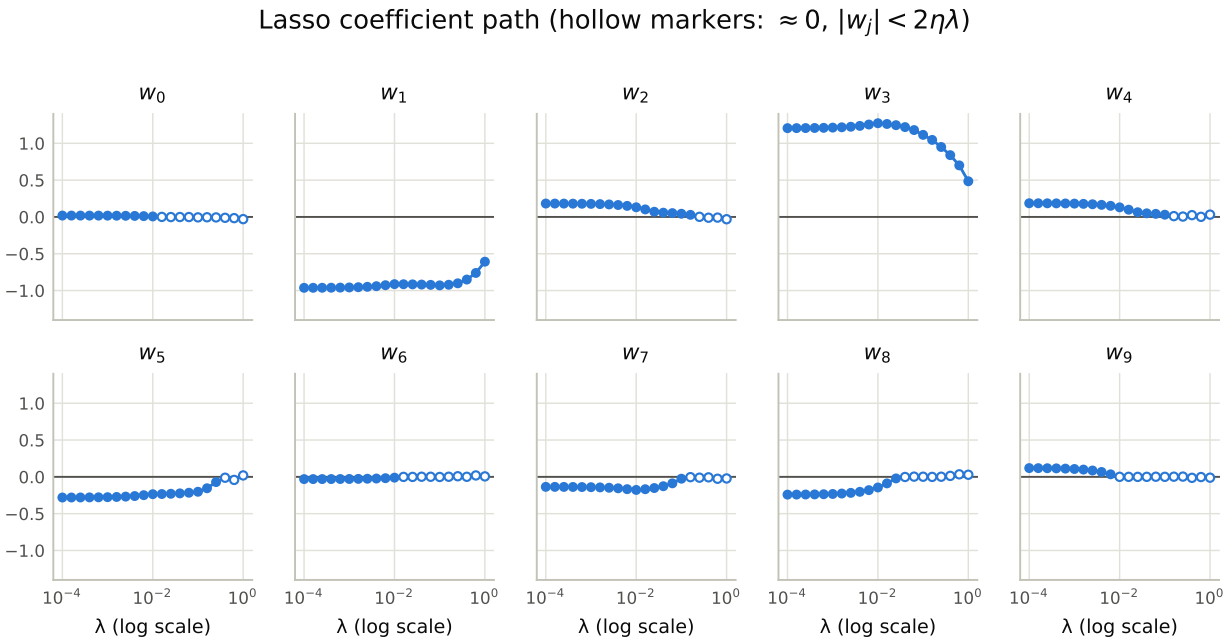


Figure 9: Lasso coefficient path; hollow markers are coefficients counted as ≈ 0 ($|w_j| < 2\eta\lambda$).

λ	$\#\approx 0$	indices j	train RMSE	test RMSE
10^{-4}	0	–	1.88×10^{-4}	0.169
1.58×10^{-4}	0	–	2.98×10^{-4}	0.168
2.51×10^{-4}	0	–	4.72×10^{-4}	0.168
3.98×10^{-4}	0	–	7.48×10^{-4}	0.168
6.31×10^{-4}	0	–	0.00119	0.167
10^{-3}	0	–	0.00188	0.165
1.58×10^{-3}	0	–	0.00298	0.163
2.51×10^{-3}	0	–	0.00472	0.160
3.98×10^{-3}	0	–	0.00748	0.156
6.31×10^{-3}	0	–	0.0119	0.152
10^{-2}	1	9	0.0183	0.146
1.58×10^{-2}	3	0, 6, 9	0.0270	0.132
2.51×10^{-2}	3	0, 6, 9	0.0376	0.119
3.98×10^{-2}	4	0, 6, 8, 9	0.0450	0.110
6.31×10^{-2}	4	0, 6, 8, 9	0.0550	0.103
10^{-1}	4	0, 6, 8, 9	0.0749	0.0985
1.58×10^{-1}	6	0, 4, 6, 7, 8, 9	0.101	0.107
2.51×10^{-1}	7	0, 2, 4, 6, 7, 8, 9	0.140	0.136
3.98×10^{-1}	8	0, 2, 4, 5, 6, 7, 8, 9	0.183	0.178
6.31×10^{-1}	8	0, 2, 4, 5, 6, 7, 8, 9	0.237	0.234
10^{0}	8	0, 2, 4, 5, 6, 7, 8, 9	0.346	0.348

Table 5: Coefficients counted as ≈ 0 ($|w_j| < 2\eta\lambda$) at each λ.

Lasso vs. Ridge. Ridge in the same Legendre basis, solved in closed form on the same grid, has its best test RMSE at $\lambda = 1.58\times10^{-1}$:

	best λ	test RMSE	$\|\mathbf{w}\|_2$	$\#\ |w_j| < 0.01$
Lasso	10^{-1}	0.09854	1.47	4
Ridge	1.58×10^{-1}	0.1422	1.44	1

Table 6: Lasso and Ridge, each at its own best λ (Legendre basis, $d = 9$).

Figure 10 shows both fits: the Lasso curve stays close to $\sin(2\pi x)$, while Ridge, which keeps sizeable w_8 and w_9, bends away from it near $x = 0$.

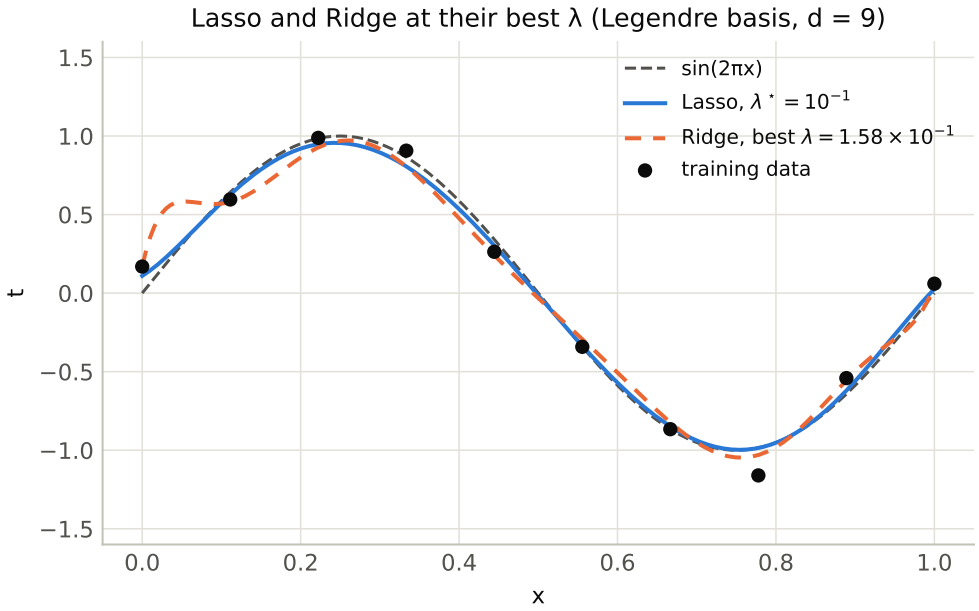


Figure 10: The Lasso fit at $\lambda^\star$ and the Ridge fit at its own best λ, with the training data and $\sin(2\pi x)$.

Discussion. L1 produces exact zeros because the subgradient of $|w_j|$ at 0 is the whole interval $[-1, 1]$: $w_j = 0$ is optimal whenever $|\mathbf{\Phi}_j^\top \mathbf{r}| \le \lambda$, so a feature whose correlation with the residual stays below λ is switched off entirely, and as λ grows the coefficients cross that threshold one after another (Figure 9, Table 5). The L2 gradient λw_j vanishes at 0, so Ridge pulls each coefficient toward zero in proportion to its size and never gets there; geometrically, the SSE contours meet the ℓ_1 ball at a corner on an axis but the round ℓ_2 ball away from the axes. Table 6 shows the result: at almost the same $\|\mathbf{w}\|_2$, the Lasso has 4 coefficients below 0.01 and Ridge only 1. The first coefficients to go carry almost no signal ($\sin(2\pi x) = -\sin(\pi u)$ has sizeable Legendre coefficients only on P_1, P_3 and P_5), so up to $\lambda^\star$ the Lasso mostly removes noise and its test RMSE falls; past $\lambda^\star$ it shrinks the signal coefficients w_1, w_3 and w_5, and train and test RMSE rise together as bias takes over.

4 P4: Bayesian regression

Method. With $\boldsymbol{\phi}(x) = [1, x, x^2, x^3]^\top$, prior $\mathcal{N}(\mathbf{0}, \alpha^{-1} I)$ and noise precision β, the posterior is $\mathcal{N}(\mathbf{w} \mid \mathbf{m}_N, S_N)$ with $S_N^{-1} = \alpha I + \beta \mathbf{\Phi}^\top \mathbf{\Phi}$ and $\mathbf{m}_N = \beta S_N \mathbf{\Phi}^\top \mathbf{t}$, and the predictive distribution is $\mathcal{N}(m(x), s^2(x))$ with $m(x) = \boldsymbol{\phi}(x)^\top \mathbf{m}_N$ and $s^2(x) = \beta^{-1} + \boldsymbol{\phi}(x)^\top S_N \boldsymbol{\phi}(x)$. We use $\alpha = 0.05$ and $\beta = 1/0.01 = 100$ for every N. S_N is never formed: with $A = S_N^{-1}$, both $\mathbf{m}_N = \beta A^{-1} \mathbf{\Phi}^\top \mathbf{t}$ and $\boldsymbol{\phi}^\top S_N \boldsymbol{\phi} = \boldsymbol{\phi}^\top A^{-1} \boldsymbol{\phi}$ come from linear solves with A.

Equivalence. Since $\mathbf{m}_N = \beta(\alpha I + \beta \mathbf{\Phi}^\top \mathbf{\Phi})^{-1} \mathbf{\Phi}^\top \mathbf{t} = (\mathbf{\Phi}^\top \mathbf{\Phi} + \frac{\alpha}{\beta} I)^{-1} \mathbf{\Phi}^\top \mathbf{t}$, the posterior mean is exactly the ridge solution with $\lambda = \alpha/\beta = 0.0005$; a Gaussian posterior's mean is also its mode, so MAP and posterior mean coincide, and so do $\mathbf{w}_{\text{MAP}}^\top \boldsymbol{\phi}(x)$ and $m(x)$. Numerically, for $N = 10$, $\mathbf{m}_N = (0.09384,\ 7.368,\ -22.81,\ 15.25)$, $\mathbf{w}_{\text{MAP}} = (0.09384,\ 7.368,\ -22.81,\ 15.25)$ and $\|\mathbf{w}_{\text{MAP}} - \mathbf{m}_N\|_2 = 1 \times 10^{-12}$: the two differ only by rounding, and their curves coincide in Figure 11.

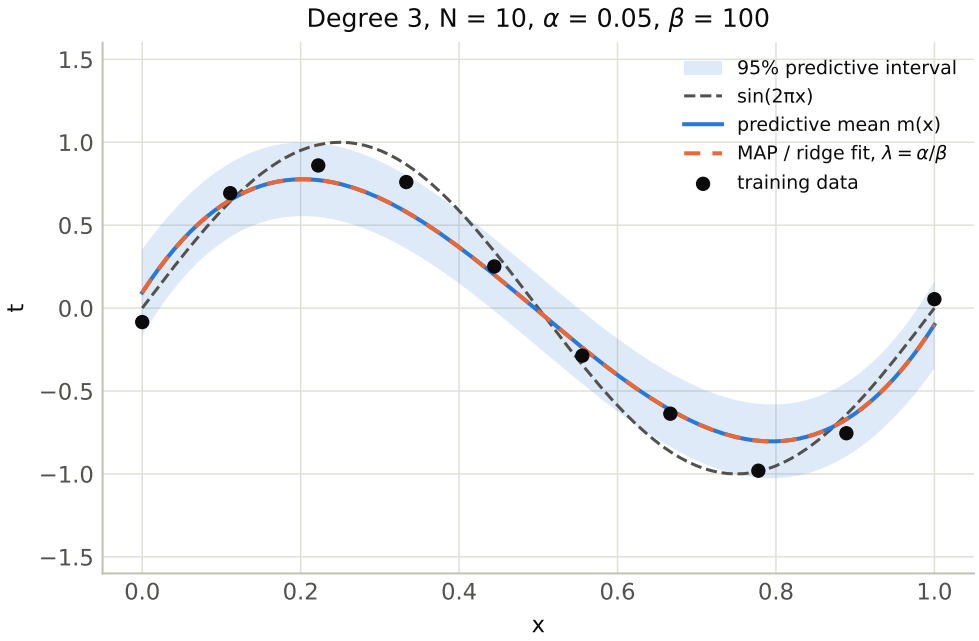


Figure 11: Posterior predictive mean and 95% interval $m(x) \pm 1.96\, s(x)$ for $N = 10$, with the MAP (ridge) fit overlaid, the training data and $\sin(2\pi x)$.

Effect of N. Table 7 summarises the interval width $2 \times 1.96\, s(x)$ for $N = 5, 10, 20, 50$, and Figure 12 plots it against x.

N	mean width	largest width	parameter share of s^2 (%)
5	0.495	0.549	37
10	0.449	0.524	24
20	0.424	0.486	14
50	0.406	0.442	6.8

Table 7: Predictive interval width over $x \in [0, 1]$, and the average share of the predictive variance that is parameter uncertainty $\boldsymbol{\phi}^\top S_N \boldsymbol{\phi}$. The noise floor is $2 \times 1.96\, \beta^{-1/2} = 0.392$.

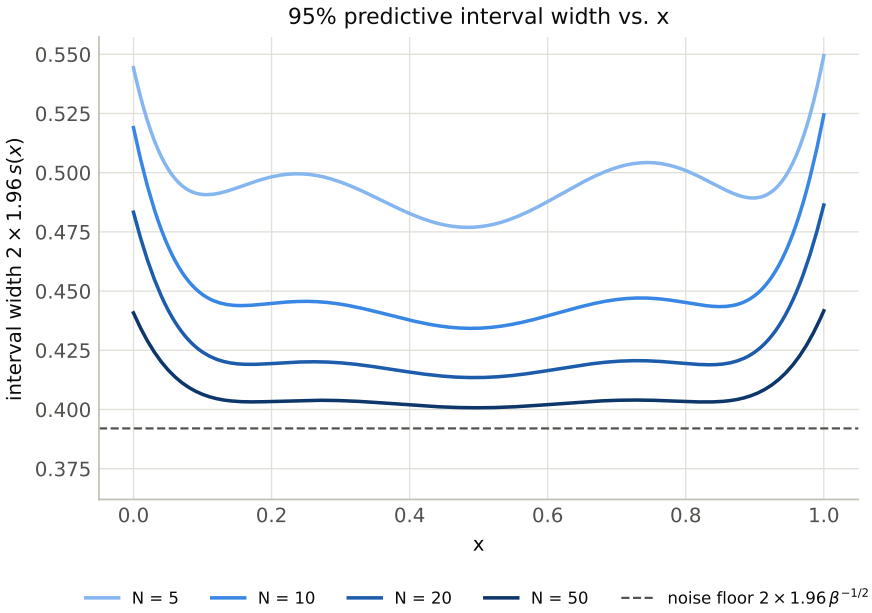


Figure 12: Width of the 95% predictive interval vs. x for each N, with the irreducible-noise floor.

Discussion. Every training point adds $\beta\boldsymbol{\phi}(x_n)\boldsymbol{\phi}(x_n)^\top$ to the posterior precision S_N^{-1}, so the posterior covariance S_N, and with it the parameter uncertainty $\boldsymbol{\phi}(x)^\top S_N \boldsymbol{\phi}(x)$, shrinks as N grows: roughly like $1/N$ in the middle of $[0, 1]$, more slowly at the ends, where a cubic is least pinned down and the band stays widest (averaged over x, its share of the predictive variance falls from 37% at $N = 5$ to 6.8% at $N = 50$). The observation-noise term β^{-1} does not depend on the data at all: it is the scatter of a new target around the true function, so no amount of data removes it, and the band can only approach the floor $2 \times 1.96\,\beta^{-1/2} = 0.392$, never cross it.